HTTP/2 Rapid Reset DoS Attack

Abstract

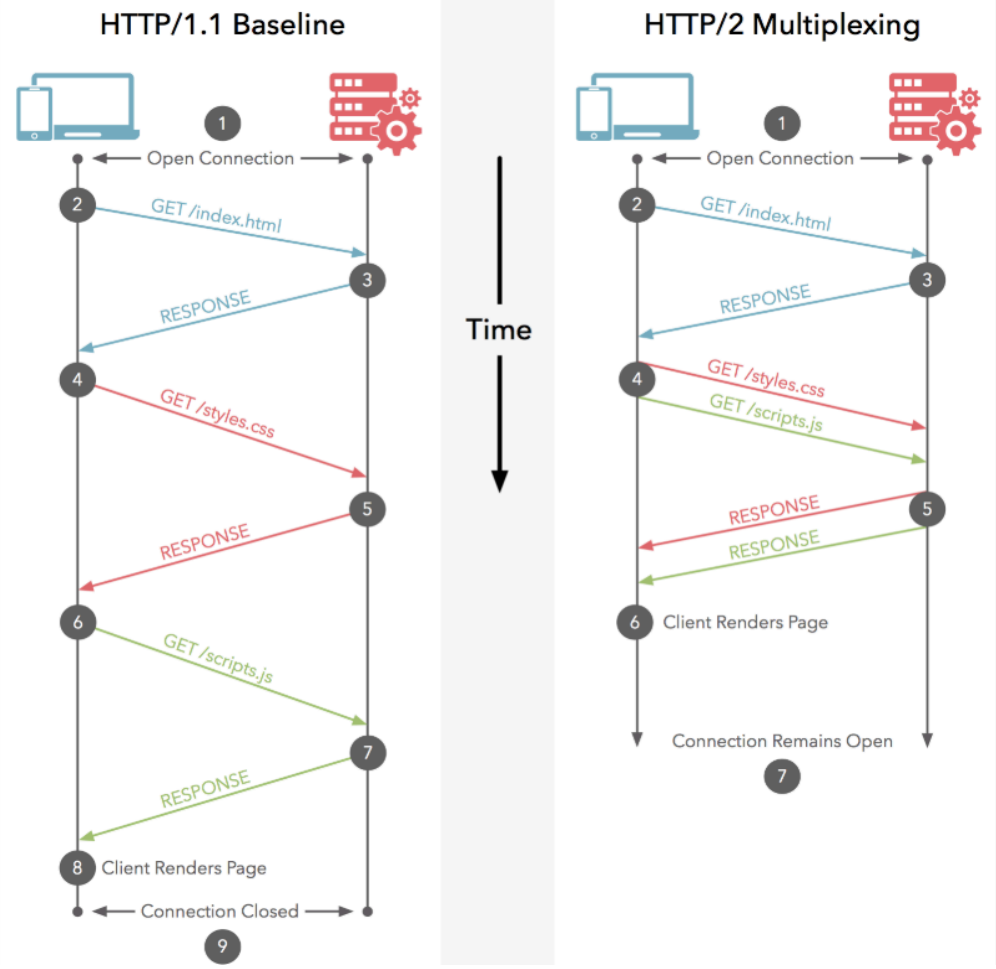
With HTTP based attacks summing up close to 51% of the total DDoS crime, shining light on the less known, more sophisticated approaches – cache busting attacks, targeted login endpoints, unusual requests, protocol violations – proves that carefully taking advantage of novel features (including the ones provided by the protocol) can break the Internet and make the headlines in the biggest's cloud and security providers' blogs. Therefore, we introduce the HTTP/2 Rapid Reset DoS Attack, matching the unusual request pattern and busting the infrastructure through intelligently crafting messages to exploit the means that make HTTP/2 more efficient.

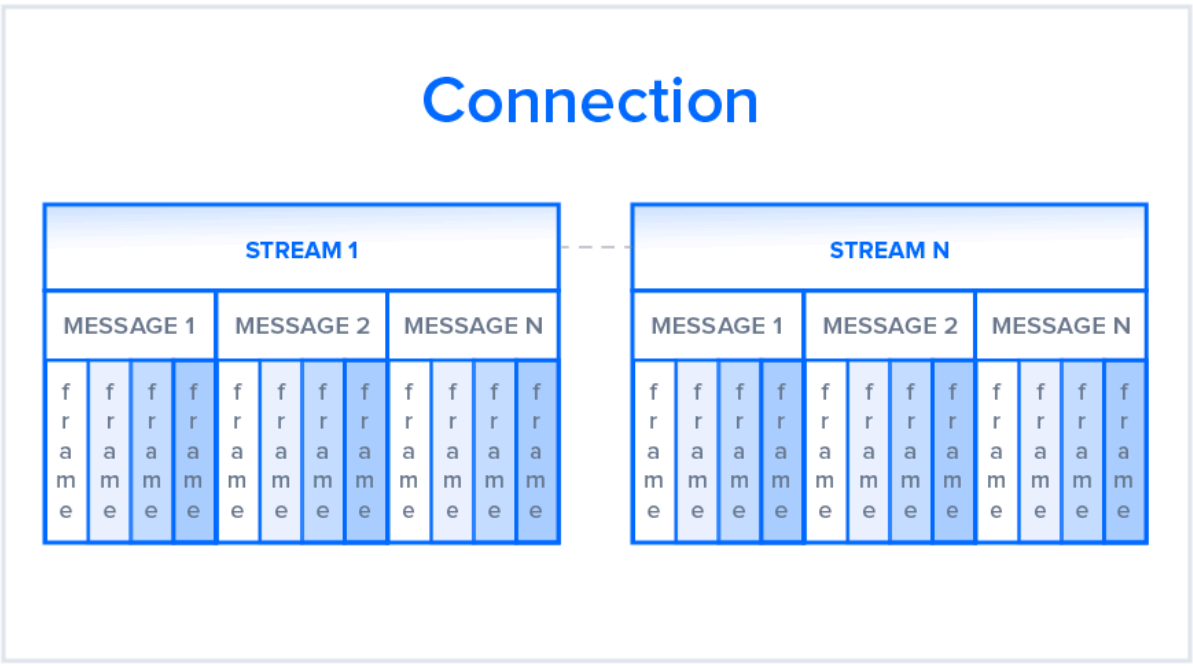
Introduction

HTTP/2 is the second most recent major version of the Hypertext Transfer Protocol, enabling better performance compared to its predecessor by binary framing, multiplexing and header compression, the first two of these features constituting the base of the HTTP/2 Rapid Reset DoS Attack. However unfortunate that is, HTTP/2 came like a great improvement by eliminating other, less sophisticated techniques of DoSing servers – head-of-line blocking, present in HTTP/1 due to the serial nature of the protocol.

When it comes to the "offending" new features, binary framing is concerned with encoding requests and responses into smaller chunks tagging each one of them to a stream sent over *one* TCP connection. Moving forward, multiplexing takes care of interlacing these tagged frames (part of either a request or a response stream) without blocking the communication channel by waiting for the whole (bigger) message to be transmitted. This technique contributes to better connection management (only one TCP connection is needed for multiple

parallel streams) and finer network utilization and bandwidth (helping with memory footprint and connection reuse). To make use of this, the client and the server communicating with each other negotiate the maximum number of concurrent streams at the beginning of the connection, while exchanging `SETTINGS` frames, with usually the server coming up with the RFC recommendation of 100 as value for `SETTINGS_MAX_CONCURRENT_STREAMS`.



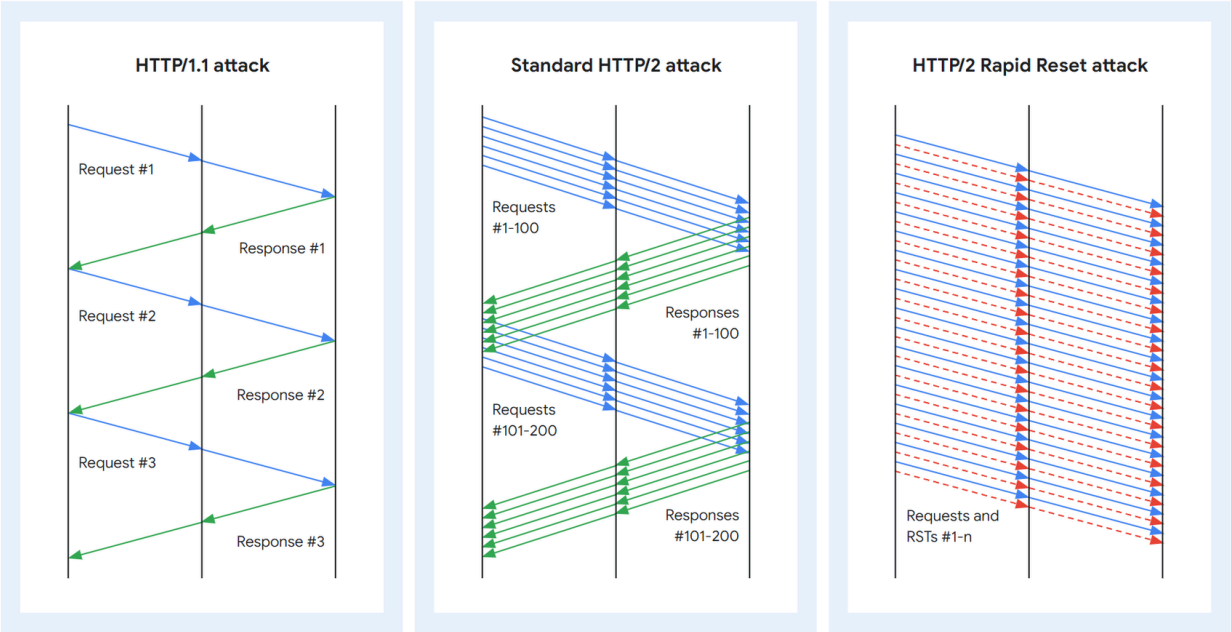


Attack description

While enforcing a maximum number of concurrent streams per TCP connection in theory offers somewhat of a protection against a DoS attack, practice has proved that backdoors exist in *everything*, even in HTTP/2.

In relation to the aforementioned connection reuse novel to HTTP/2, the protocol adds a new, special frame under the civil name of `RST_STREAM`, allowing a client to change its mind and drop an already sent request without the penalty of opening a new connection and losing all the precious bandwidth, essentially taking out the reset stream from the internal counter being compared to `SETTINGS_MAX_CONCURRENT_STREAMS`. On one hand, the client can proceed as it wishes after such a frame leaves its end, because the protocol makes the cancellation unilateral. On the other hand however, the server has to drop all the context it built for the reset stream and construct a new one for the cancellation stream, essentially employing much more computing power to address the situation than the counterpart.

This asymmetry, and the free real estate provided by the cancellation, are the major drivers for the HTTP/2 Rapid Reset DoS Attack: the attacker sends as many requests the network can sustain and cancels them immediately, *not having one response to handle* (and spend computing power on), while the victim server is drowning in revoking streams and making room for new ones, all this happening on a single TCP connection!



Related attacks

Mitigations employed to address the attack led to two main HTTP/2 Rapid Reset mutations, able to bypass security measures but much more tame given the new state-of-practice.

The first one focused on going past the limit employed by some servers on the number of `RST_STREAMS` frames a client could use per connection by opening up batches of bogus streams (less than the upper threshold for the number of resets per connection) and resetting them after a while. Rinse and repeat, with the “disadvantage” that such an approach does not take full advantage of the connection.

The second one concentrates on sending as many requests as possible, ignoring the limit imposed by the server and giving up on resetting the frames. This takes advantage of servers not closing a connection when the limit is reached, but rather queueing and responding to the message once a free spot is available, following a relaxed interpretation of the protocol which states that exceeding streams should only be invalidated, without losing the connection.

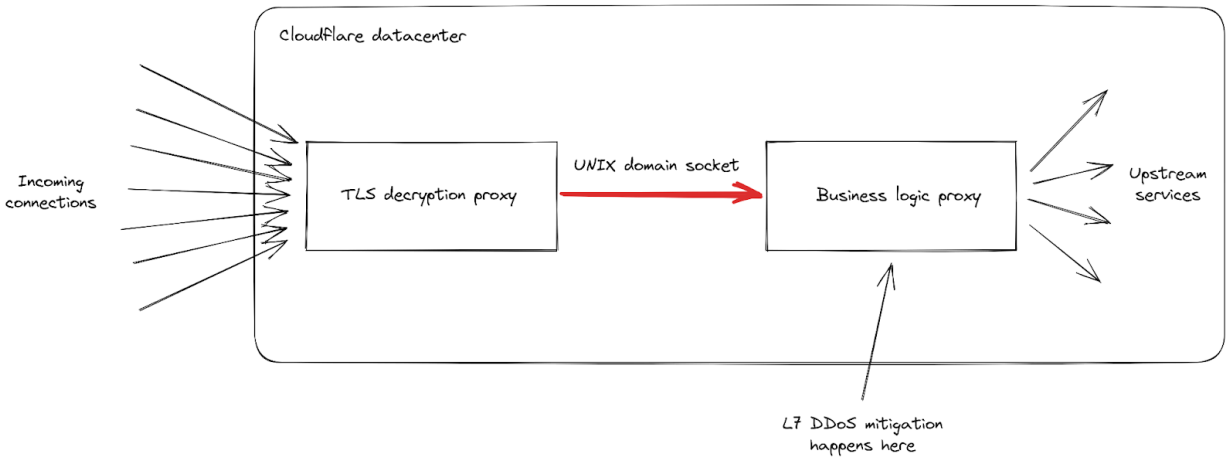
Effects, attack mitigation

Effects

The effects of the HTTP/2 Rapid Reset Attack live full-time in the application layer (L7) as this is the level where the flooding takes place, targeting the apps backend’s ability to accept and parse new requests.

In the wild, Cloudflare's clients observed a couple of different HTTP errors, related to different parts of their security architecture being exhausted with messages:

- `502`s – Cloudflare's TLS decryption proxy is overwhelmed with the sheer number of requests and is unable to forward the processed frames to the actual DDoS detection engine.



- 499s – Cloudflare's mitigation to the HTTP/2 DoS attacks clamped the number of concurrent streams resets to 64, closing the connection on legitimate clients that went the fast connection establish route without checking the server limit and guessing the upper threshold to be the recommended protocol default of 100. When resetting such streams, legitimate clients triggered the mitigation system and were met with a sudden connection reset error.

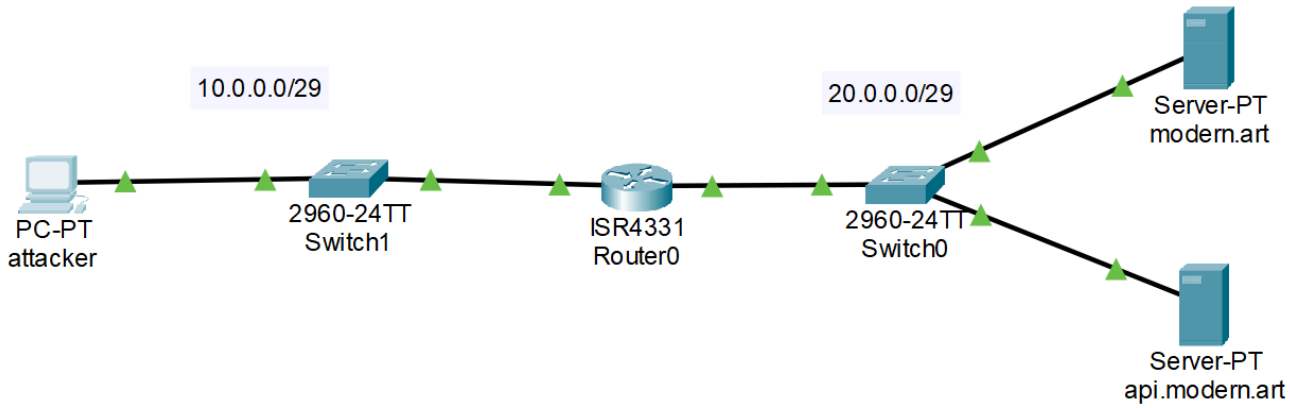
Mitigation

To protect against `RST_STREAM` vulnerabilities, servers can implement the following technical and architectural safeguards[8]:

- **Discard Resets on Idle Streams**: servers should be configured to ignore `RST_STREAM` frames received on streams that are currently idle. If a stream has not yet sent a request, a reset command is logically invalid and should be treated as such.
- **Enforce HTTPS (TLS) Encryption**: Mandating HTTPS prevents attackers from inspecting traffic to identify requested resources. Furthermore, TLS encryption stops third parties from injecting forged frames into the data stream.
- **Prioritize Critical Asset Loading**: Structure the application to load essential resources as early as possible in the page lifecycle. This strategy reduces the "window of vulnerability" during which an attacker can disrupt the loading process.
- **Authenticate Reset Frames** (Theoretical): While this requires extending the current HTTP/2 protocol, a robust defense involves authenticating `RST_STREAM` frames. This ensures the command originates from the legitimate server rather than an external attacker.
- **Await Protocol Standardization**: The ultimate resolution lies in updating the HTTP/2 specification itself. Industry efforts are currently underway to amend the standard, providing implementers with clear, built-in guidance to prevent these attacks.

Proof-of-Concept description and implementation

The Proof-of-Concept was implemented using a custom network topology within Containernet[1] in order to ensure complete control over the underlying infrastructure.



This setup replicates a real-world scenario in which an attacker targets and overwhelms a remote server situated in a cloud infrastructure. The 3 main components of the topology are as follow:

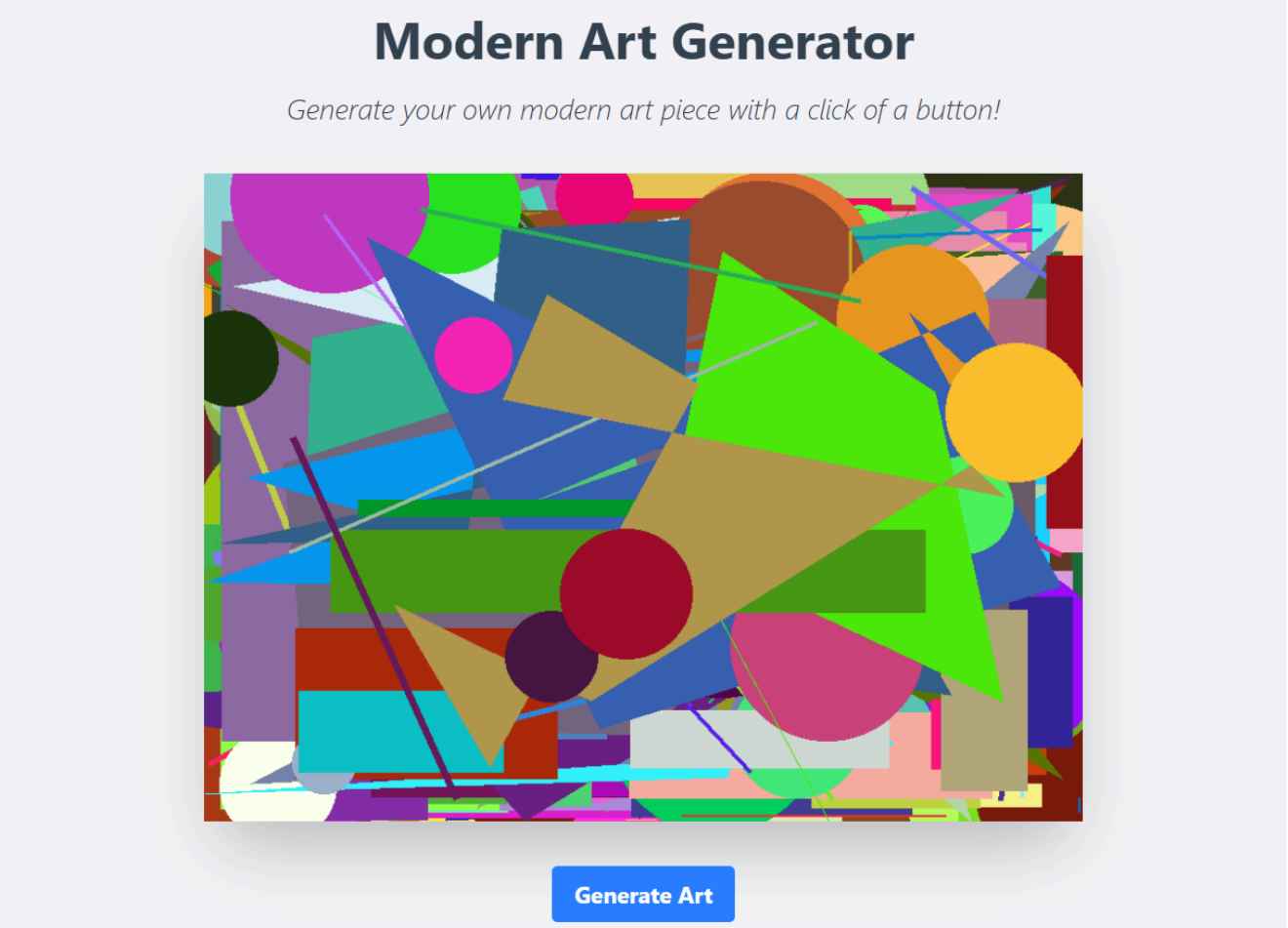
1. Attacker

Utilizes a custom-made Golang script to spawn numerous workers that open HTTP/2 connection in conformity with the attack described above. It has 4 arguments:

- url - the url of the website
- connections - number of connections to open
- requests - number of requests to send for each connection
- concurrency - maximum number of workers spawned

2. NGINX web server + proxy

Servers the static HTML site and acts as a reverse proxy to the backend and is the component being targeted by the attack, as it's the one handling the HTTP/2 connections (as part of HTTPS).



In order to ensure that the attack had the best chance of succeeding, we used an older version of NGINX - 1.23, which lacks the patch[2] setting an upper bound for the number of new streams that can be created in an event loop, as well as increasing the default values for the following fields:

- `http2_max_concurrent_streams`
- `keepalive_requests`
- `keepalive_timeout`

3. FastAPI business logic

This component handles the creation of the “art pieces” by randomly placing geometric figures on a blank canvas. Within the scope of the attack, this component acts as a force multiplier for resource consumption. It burdens the NGINX service by compelling it to manage SSL termination and proxying overheads while simultaneously handling deliberately delayed backend responses.
It exposes two endpoints:

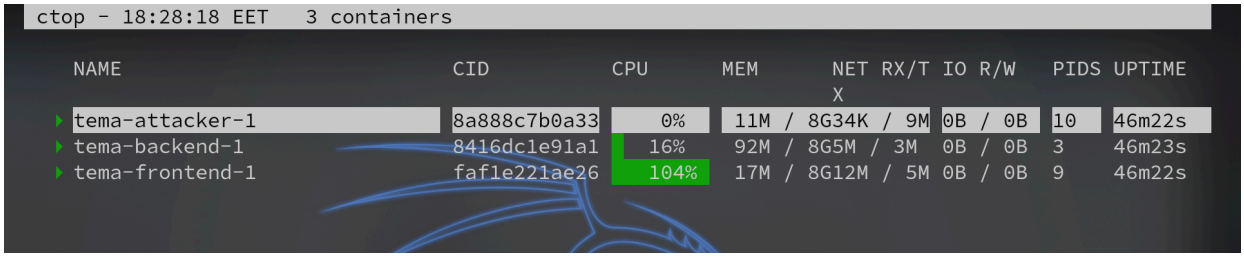
- `/api/image` - the image generation endpoint
- `/api/health` - health check for the service

Results

In order to better visualize the effects of the attack, we will use *ctop*[3], allowing us to see in real-time the resources used by active Docker containers including memory, cpu usage, network statistics and more.
Starting the topology and entering the attacker container, we can start the attack by running the following command:

```
./attacker -url='https://modern.art:443/api' -requests=500000
```

Under a load of 500,000 concurrent requests, the target server experienced immediate CPU exhaustion, with usage levels reaching 104%.



The attack successfully caused a denial of service, evidenced by the persistent `curl: (28) SSL connection timeout` error when attempting to access the site.

References

1. Containernet | Use Docker containers as hosts in Mininet emulations.
2. HTTP/2 Rapid Reset Attack Impacting NGINX Products - NGINX
3. ctop
4. Record-breaking 5.6 Tbps DDoS attack and global DDoS trends for 2024 Q4
5. HTTP/1.1 vs HTTP/2: What's the Difference?
6. How it works: The novel HTTP/2 'Rapid Reset' DDoS attack
7. HTTP/2 Rapid Reset: deconstructing the record-breaking attack
8. https://owasp.org/www-community/attacks/HTTP2_Reset_Attack